
Building Redis from Scratch

in Modern C++17

A complete zero-to-production guide
Codecrafters Project — Interview Ready Edition

TCP Sockets

RESP Protocol

Concurrency

Persistence

Language	Standard	Platform
C++	C++17	Linux / macOS

[Sockets](#) · [Parsing](#) · [Expiry](#) · [RDB Persistence](#) · [Replication](#) · [Interview Q&A](#)

Contents

1	Introduction & Project Overview	4
1.1	What Is This Project?	4
1.2	What Is Redis (Conceptually)?	4
1.3	Skills You Will Gain	5
2	Environment Setup	6
2.1	Prerequisites	6
2.1.1	Installation (Ubuntu / Debian)	6
2.1.2	Installation (macOS)	7
2.2	Git — Everything You Need to Know	7
2.2.1	Core Git Concepts	7
2.2.2	Essential Git Commands for This Project	7
2.3	Project Directory Structure	8
2.3.1	CMakeLists.txt — Explained	9
3	Networking Fundamentals	10
3.1	The OSI Model — Big Picture	10
3.2	TCP — Transmission Control Protocol	10
3.2.1	3-Way Handshake	11
3.3	Sockets — The POSIX API	11
4	Step 1 — Minimal TCP Server	12
4.1	Goal	12
4.2	Full Code	12
4.3	Line-by-Line Explanation	13
4.3.1	#include headers	13
4.3.2	socket(AF_INET, SOCK_STREAM, 0)	13
4.3.3	setsockopt(...SO_REUSEADDR...)	14
4.3.4	struct sockaddr_in addr	14
4.3.5	listen(server_fd, 5)	14
4.3.6	accept(server_fd, nullptr, nullptr)	14
4.4	Testing Stage 1	14
5	The RESP Protocol	16
5.1	What is RESP?	16
5.2	RESP Data Types	16
5.3	Anatomy of a RESP Command	16
5.4	RESP Parser — Full Implementation	17

5.4.1	Parser — Line-by-Line	18
6	Step 2 — Handling Multiple Clients	19
6.1	The Problem with a Single-Client Loop	19
6.2	Multi-Client Server	19
6.2.1	Key Concepts — Threading	21
7	Step 3 — Core Commands	22
7.1	Command Dispatcher	22
7.2	Command Explanations	24
7.2.1	PING	25
7.2.2	ECHO	25
7.2.3	SET & GET	25
7.2.4	EXISTS	25
7.2.5	DEL	25
7.2.6	INCR	25
8	Step 4 — Key Expiry (TTL)	27
8.1	Concept	27
8.2	Approach: Lazy Expiry	27
8.3	Implementation	27
8.4	Chrono — C++ Time Library	29
9	Step 5 — RDB Persistence	30
9.1	What is RDB?	30
9.2	RDB File Structure	30
9.3	RDB Parser — C++ Implementation	31
9.4	RDB — Key Concepts	33
9.4.1	Length Encoding	33
9.4.2	Endianness in RDB	33
9.5	Command-Line Arguments	33
10	Step 6 — Replication	35
10.1	Replication Overview	35
10.2	Replication Handshake	35
10.3	Replication ID and Offset	36
10.4	Primary-Side Implementation	36
10.5	Replica-Side: Connecting to Primary	37
11	Step 7 — Complete Server	39
11.1	Final <code>server.cpp</code>	39
12	Interview Preparation	46
12.1	System Design Questions	46
12.2	C++ Systems Programming Questions	47
12.3	Data Structure Questions	48
13	Concepts Glossary	49
14	Cheat Sheets	51

14.1	RESP Encoding Reference	51
14.2	Redis Commands — Quick Reference	51
14.3	Build & Test Commands	52
14.4	Common Errors and Fixes	52
A	Debugging with GDB	54
B	Valgrind — Memory Error Detection	55
C	Further Reading	56

Chapter 1

Introduction & Project Overview

1.1 What Is This Project?

Project Goal

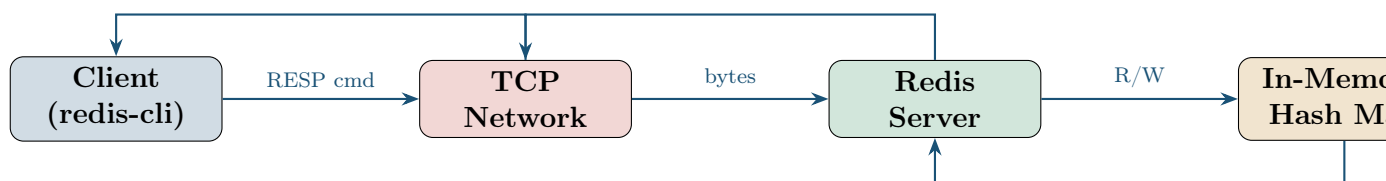
You will build a **working Redis server** in C++ from absolute zero — TCP listener, RESP parser, command engine, key expiry, RDB persistence and basic replication — exactly as done on the Codecrafters platform.

Redis (*Remote Dictionary Server*) is one of the most widely deployed in-memory databases in the world. It is used by Twitter, GitHub, Snapchat, Stack Overflow and thousands of other companies. By rebuilding it from scratch you will deeply understand:

- How a **TCP server** accepts and manages connections.
- How **binary protocols** encode commands and replies.
- How **key-value stores** implement expiry, persistence and replication.
- Low-level C++ socket programming — exactly the kind of knowledge that separates *senior* engineers from the rest.

1.2 What Is Redis (Conceptually)?

Redis is a **single-threaded, in-memory, key-value store** that listens on TCP port **6379** by default.



Key properties of Redis:

Property	Meaning
In-memory	Data lives in RAM → microsecond latency
Single-threaded event loop	One thread handles all clients (multiplexed with <code>poll</code> / <code>epoll</code>)
Persistent optional	RDB snapshots & AOF logs save data to disk
Replication	One primary, many replicas (leader–follower)
Rich data types	strings, lists, sets, sorted sets, hashes, streams

1.3 Skills You Will Gain

What you will know after finishing

1. POSIX socket API — `socket()`, `bind()`, `listen()`, `accept()`, `send()`, `recv()`
2. RESP (Redis Serialization Protocol) — binary framing
3. Multithreading in C++17 (`std::thread`)
4. Hash-map internals (`std::unordered_map`)
5. Monotonic clock for TTL expiry
6. RDB binary file format parsing
7. Leader–follower replication handshake
8. How to read and explain every line of systems code in an interview

Chapter 2

Environment Setup

2.1 Prerequisites

You need a Linux or macOS machine (or WSL2 on Windows).

Tool	Version	Why
<code>g++</code> or <code>clang++</code>	≥ 9	C++17 support
<code>cmake</code>	≥ 3.13	Build system
<code>git</code>	any	Source control
<code>redis-cli</code>	any	Test your server
<code>nc</code> (netcat)	any	Low-level socket testing

2.1.1 Installation (Ubuntu / Debian)

Install dependencies

```
1  # Update package lists
2  sudo apt update
3
4  # Install C++ compiler, cmake, git, netcat
5  sudo apt install -y g++ cmake git netcat-openbsd
6
7  # Install Redis client (for testing)
8  sudo apt install -y redis-tools
9
10 # Verify installations
11 g++ --version           # should print g++ 9+ or higher
12 cmake --version        # should print 3.13+
13 git --version           # any version fine
14 redis-cli --version     # any version fine
```


2.1.2 Installation (macOS)

macOS (Homebrew)

```

1 # Install Homebrew if you don't have it
2 /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
3
4 # Install tools
5 brew install cmake git redis
6
7 # macOS ships clang++ with Xcode Command Line Tools
8 xcode-select --install

```

2.2 Git — Everything You Need to Know

What is Git?

Git is a **distributed version control system**. It tracks every change you make to source files, lets you revert mistakes, and allows collaboration. For Codecrafters, every stage is submitted via `git push`.

2.2.1 Core Git Concepts

Term	Meaning
Repository (repo)	A directory tracked by Git; contains a hidden <code>.git/</code> folder
Commit	A snapshot of all tracked files at a moment in time
Branch	A movable pointer to a commit (default: <code>main</code>)
Remote	A copy of the repo on a server (GitHub, Codecrafters)
Working tree	Files you can see and edit right now
Staging area	Files queued for the next commit (<code>git add</code>)
HEAD	The commit currently checked out

2.2.2 Essential Git Commands for This Project

Git workflow

```

1 # Clone the Codecrafters starter repo
2 git clone https://codecrafters.io/redis-starter-cpp my-redis
3 cd my-redis
4
5 # After every stage: add, commit, push
6 git add .
7 git commit -m "feat: implement PING command"

```

```
8 git push origin main
9
10 # Check what changed
11 git status          # show modified files
12 git diff            # show exact line changes
13 git log --oneline   # show commit history compactly
14
15 # Undo last commit (keep changes)
16 git reset HEAD~1
```

What Every Git Command Does — Line by Line

1. `git add .` — Moves *all* changed files from the working tree into the **staging area**. The dot (`.`) means "everything in the current directory."
2. `git commit -m "message"` — Takes a permanent snapshot of the staged files. The `-m` flag lets you write the commit message inline. Always write *imperative* messages: "Add GET command", not "added GET."
3. `git push origin main` — Uploads all local commits to the remote named `origin` (Codecrafters/GitHub) on the `main` branch. Codecrafters' CI runs your tests on push.
4. `git status` — Shows which files are untracked, staged, or modified. Run this constantly.
5. `git log -oneline` — Compact history. Each line is one commit: `<hash> <message>`.

Interview tip — Git

Common interview questions:

- "What is the difference between *git merge* and *git rebase*?"
Merge creates a merge commit, preserving history. **Rebase** replays commits on top of another branch — linear, cleaner history, but rewrites commits.
- "What does *git stash* do?"
Saves your uncommitted work temporarily so you can switch branches, then applies it back with `git stash pop`.

2.3 Project Directory Structure

Folder layout

```
1 my-redis/
2 +-- CMakeLists.txt      # Build configuration
3 +-- src/
4 |   +-- server.cpp      # ALL our code lives here
5 +-- .codecrafters.yml   # Codecrafters build config
6 +-- README.md
```

2.3.1 CMakeLists.txt — Explained

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.13)
2 # ^ Tell CMake the oldest version we support
3
4 project(redis-starter-cpp)
5 # ^ Name our project
6
7 set(CMAKE_CXX_STANDARD 17)
8 # ^ Require C++17 features (structured bindings, std::
   optional\ldots{ })
9
10 add_executable(server src/server.cpp)
11 # ^ Compile server.cpp into an executable called "server"
12
13 target_compile_options(server PRIVATE -Wall -Wextra)
14 # ^ Enable all warnings --- catches bugs early
```

How to Build and Run

Build commands

```
1 mkdir -p build && cd build      # Create build directory
2 cmake ..                        # Generate Makefiles
3 make -j$(nproc)                  # Compile (all CPU cores)
4 ./server                         # Run your Redis server
```

Chapter 3

Networking Fundamentals

3.1 The OSI Model — Big Picture

Why networking matters

Every time a client sends `PING` to your server, data travels through a network **stack**. Understanding at least the TCP layer is mandatory for systems programming interviews.

Layer 7 — Application (Redis / HTTP / DNS)

Layer 4 — Transport (TCP / UDP)

Layer 3 — Network (IP)

Layer 2 — Data Link (Ethernet)

Layer 1 — Physical (Cables / WiFi)

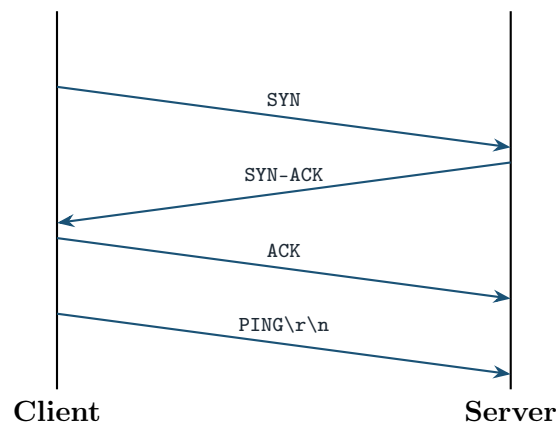
We work at **Layer 7** (Redis protocol) riding on top of **Layer 4** (TCP). The OS handles everything below that for us.

3.2 TCP — Transmission Control Protocol

TCP in one paragraph

TCP is a **reliable, connection-oriented** protocol. Before any data flows, the two sides perform a **3-way handshake** (`SYN` → `SYN-ACK` → `ACK`). TCP guarantees delivery, ordering, and error-checking. Redis *requires* TCP because a dropped byte would corrupt a command.

3.2.1 3-Way Handshake



3.3 Sockets — The POSIX API

A **socket** is a file descriptor (an integer) that represents one endpoint of a TCP connection. The POSIX socket API is the universal interface for network programming on Linux/macOS.

Function	Returns	What it does
<code>socket()</code>	fd (int)	Create a socket file descriptor
<code>setsockopt()</code>	0 or -1	Set socket options (e.g. reuse port)
<code>bind()</code>	0 or -1	Attach socket to an IP:port
<code>listen()</code>	0 or -1	Mark socket as passive (accepting)
<code>accept()</code>	fd (int)	Block until a client connects, return new fd
<code>recv()</code>	bytes read	Read bytes from connected fd
<code>send()</code>	bytes sent	Write bytes to connected fd
<code>close()</code>	0 or -1	Destroy the socket

Interview tip — Sockets

"What is the difference between `socket()`, `bind()`, `listen()`, and `accept()`?"

- `socket()` — creates the socket (like buying a phone)
- `bind()` — assigns address/port (getting a phone number)
- `listen()` — starts watching for callers
- `accept()` — picks up one caller and returns a new fd

Chapter 4

Step 1 — Minimal TCP Server

4.1 Goal

Accept a TCP connection on port 6379, send `+PONG\r\n` back, then close. This is the very first Codecrafters stage.

4.2 Full Code

src/server.cpp -- Stage 1: TCP + PING

```
1  #include <iostream>
2  #include <string>
3  #include <cstring>
4  #include <sys/socket.h>
5  #include <netinet/in.h>
6  #include <unistd.h>
7
8  using namespace std;
9
10 int main() {
11     // 1. Create a TCP socket (IPv4, stream, auto-protocol)
12     int server_fd = socket(AF_INET, SOCK_STREAM, 0);
13     if (server_fd < 0) {
14         cerr << "socket() failed\n";
15         return 1;
16     }
17
18     // 2. Allow reuse of port --- avoids "Address already
19         in use"
20     int opt = 1;
21     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR,
22                 &opt, sizeof(opt));
23
24     // 3. Describe the address we want to bind to
25     struct sockaddr_in addr;
26     addr.sin_family = AF_INET;    // IPv4
```

```

26     addr.sin_port          = htons(6379); // port in network
        byte order
27     addr.sin_addr.s_addr = INADDR_ANY; // listen on all
        interfaces
28
29     // 4. Bind the socket to that address
30     if (bind(server_fd, (struct sockaddr*)&addr, sizeof(
        addr)) < 0) {
31         cerr << "bind() failed\n";
32         return 1;
33     }
34
35     // 5. Start listening --- queue up to 5 pending
        connections
36     listen(server_fd, 5);
37     cout << "Listening on port 6379...\n";
38
39     // 6. Accept ONE client connection (blocking call)
40     int client_fd = accept(server_fd, nullptr, nullptr);
41
42     // 7. Send a raw RESP Simple String reply: +PONG\r\n
43     string pong = "+PONG\r\n";
44     send(client_fd, pong.c_str(), pong.size(), 0);
45
46     // 8. Close both sockets cleanly
47     close(client_fd);
48     close(server_fd);
49     return 0;
50 }

```

4.3 Line-by-Line Explanation

4.3.1 #include headers

Header	What it provides
<code><sys/socket.h></code>	<code>socket()</code> , <code>bind()</code> , <code>listen()</code> , <code>accept()</code> , <code>send()</code> , <code>recv()</code>
<code><netinet/in.h></code>	<code>sockaddr_in</code> , <code>INADDR_ANY</code> , <code>AF_INET</code> , <code>htons()</code>
<code><unistd.h></code>	<code>close()</code> , <code>read()</code> , <code>write()</code>
<code><cstring></code>	<code>memset()</code> , <code>memcpy()</code>

4.3.2 `socket(AF_INET, SOCK_STREAM, 0)`

- `AF_INET` — Address Family Internet (IPv4). Use `AF_INET6` for IPv6.
- `SOCK_STREAM` — TCP (reliable, ordered byte stream). `SOCK_DGRAM` would give UDP.

- `0` — Let the OS choose the protocol automatically (TCP for `SOCK_STREAM`).
- **Returns** an *integer* file descriptor, just like `open()` for files. `-1` means error.

4.3.3 `setsockopt(...SO_REUSEADDR...)`

Without this, if you restart your server quickly, the OS still holds the port open for 60 seconds (`TIME_WAIT` state) and `bind()` fails with "Address already in use." Setting `SO_REUSEADDR` tells the kernel: "let me reuse this port immediately."

4.3.4 `struct sockaddr_in addr`

Field	Value	Meaning
<code>sin_family</code>	<code>AF_INET</code>	IPv4
<code>sin_port</code>	<code>htons(6379)</code>	Port in <i>network byte order</i>
<code>sin_addr.s_addr</code>	<code>INADDR_ANY</code>	Accept from any IP address

What is `htons()`?

Computers store multi-byte integers in either **little-endian** (x86 — least significant byte first) or **big-endian** (network — most significant byte first). `htons()` = "host to network short" — converts your CPU's format to network format for a 16-bit value. `htonl()` does the same for 32-bit.

4.3.5 `listen(server_fd, 5)`

The second argument is the **backlog** — how many pending connection requests the kernel should queue before refusing new ones. 5 is standard for simple servers. For high-traffic servers, use 128 or more.

4.3.6 `accept(server_fd, nullptr, nullptr)`

`accept()` **blocks** (pauses the program) until a client connects. When one does, it returns a *new* file descriptor dedicated to that client. The original `server_fd` stays open to accept more clients. The last two args can hold the client's address — we pass `nullptr` since we don't need it yet.

4.4 Testing Stage 1

Test your server

```

1 # Terminal 1: Build and run
2 cd build && make && ./server
3
4 # Terminal 2: Connect with redis-cli
5 redis-cli ping
6 # Expected: PONG

```



```
7  
8 # Or with netcat (raw TCP)  
9 echo -e "*1\r\n$4\r\nPING\r\n" | nc localhost 6379  
10 # Expected: +PONG
```

Chapter 5

The RESP Protocol

5.1 What is RESP?

RESP — Redis Serialization Protocol

RESP is the **binary-safe text protocol** Redis uses for client-server communication. It is simple enough to parse in ≈ 50 lines of C++, yet expressive enough to represent all Redis data types. Every message ends with `\r\n` (CRLF).

5.2 RESP Data Types

Symbol	Type	Meaning	Example	Use
+	Simple String	Plain text (no <code>\r\n</code> inside)	<code>+OK\r\n</code>	SET
-	Error	Error message	<code>-ERR unknown\r\n</code>	Bad c
:	Integer	64-bit signed integer	<code>:42\r\n</code>	INCR
\$	Bulk String	Binary-safe, length-prefixed	<code>\$4\r\nPING\r\n</code>	Most
*	Array	Array of RESP values	<code>*2\r\n\$3\r\nGET\r\n\$3\r\nfoo\r\n</code>	Comm

5.3 Anatomy of a RESP Command

When you type `SET key value` in redis-cli, it sends:

RESP wire format for SET key value

```
1 *3\r\n      <- Array of 3 elements
2 $3\r\n      <- Bulk string of length 3
3 SET\r\n      <- The string "SET"
4 $3\r\n      <- Bulk string of length 3
5 key\r\n      <- The string "key"
6 $5\r\n      <- Bulk string of length 5
7 value\r\n      <- The string "value"
```

Why length-prefix instead of newline-delimited?

A length prefix lets the value itself contain `\r\n` without confusing the parser. This is **binary-safe**: you can store any bytes, including null bytes or newlines, as a Redis value.

5.4 RESP Parser — Full Implementation

RESP parser in C++17

```

1  #include <vector>
2  #include <string>
3  #include <stdexcept>
4
5  using namespace std;
6
7  // Parse a RESP array (the only shape a client sends
8  // commands in).
9  // Returns vector of strings: ["SET", "key", "value"]
10 vector<string> parseRESP(const string& raw) {
11     vector<string> args;
12     size_t pos = 0;                                // current read
13                                                     position
14
15     // Commands are always RESP Arrays starting with '*'
16     if (raw[pos] != '*') {
17         throw runtime_error("Expected array");
18     }
19     pos++;                                          // skip '*'
20
21     // Read the element count (e.g. "3" from "*3\r\n")
22     size_t crlf = raw.find("\r\n", pos);
23     int count = stoi(raw.substr(pos, crlf - pos));
24     pos = crlf + 2;                                // skip past "\r\n"
25     "
26
27     for (int i = 0; i < count; i++) {
28         // Each element is a bulk string: $<len>\r\n<data>\r\n
29         if (raw[pos] != '$') {
30             throw runtime_error("Expected bulk string");
31         }
32         pos++;                                      // skip '$'
33
34         crlf = raw.find("\r\n", pos);
35         int len = stoi(raw.substr(pos, crlf - pos));
36         pos = crlf + 2;                                // skip "\r\n"
37         "after length
38
39         args.push_back(raw.substr(pos, len));

```

```
36         pos += len + 2;                                // skip data +
           trailing "\r\n"
37     }
38     return args;                                        // e.g. {"SET", "
           key", "value"}
39 }
```

5.4.1 Parser — Line-by-Line

1. **Line 13** `if (raw[pos] != '*')` — Every client command is a RESP array, starting with `*`. If not, bail with an exception.
2. **Line 16–19** `find("\r\n", pos)` — Scan forward until we find CRLF. The text between `*` and CRLF is the element count. `stoi` converts the string to an integer.
3. **Line 23–24** Inner loop runs `count` times — once per argument.
4. **Line 26** Check for `$` prefix of a bulk string.
5. **Line 31** `int len = stoi(...)` — Parse the declared byte length of the upcoming string.
6. **Line 34** `raw.substr(pos, len)` — Extract exactly `len` bytes as the argument string.
7. **Line 35** `pos += len + 2` — Advance past the string data *and* its trailing `\r\n`.

Chapter 6

Step 2 — Handling Multiple Clients

6.1 The Problem with a Single-Client Loop

Stage 1 closed after one client. Real Redis must handle *thousands* of concurrent connections. The simplest approach for our project: **one thread per client**.

std::thread in C++17

`std::thread` launches a function in a **parallel OS thread**. Each thread has its own stack but shares the process heap (global variables, heap allocations).

6.2 Multi-Client Server

Multi-threaded accept loop

```
1  #include <iostream>
2  #include <string>
3  #include <thread>
4  #include <unordered_map>
5  #include <mutex>
6  #include <cstring>
7  #include <sys/socket.h>
8  #include <netinet/in.h>
9  #include <unistd.h>
10
11  using namespace std;
12
13  // Shared in-memory store --- protected by a mutex
14  unordered_map<string, string> store;
15  mutex store_mutex;
16
17  // Forward-declared functions (implemented later)
18  vector<string> parseRESP(const string& raw);
19  string handleCommand(const vector<string>& args);
20
21  // Handle one client in its own thread
22  void handleClient(int client_fd) {
```

```

23     char buf[4096];
24
25     while (true) {
26         // Read bytes sent by the client
27         ssize_t n = recv(client_fd, buf, sizeof(buf) - 1,
28             0);
29
30         if (n <= 0) break;    // client disconnected or
31                               error
32
33         buf[n] = '\0';        // null-terminate for safety
34         string raw(buf, n);   // wrap in std::string
35
36         // Parse and execute the RESP command
37         try {
38             auto args = parseRESP(raw);
39             string reply = handleCommand(args);
40             send(client_fd, reply.c_str(), reply.size(), 0)
41             ;
42         } catch (const exception& e) {
43             string err = "-ERR " + string(e.what()) + "\r\n"
44             ;
45             send(client_fd, err.c_str(), err.size(), 0);
46         }
47     }
48     close(client_fd); // clean up when client disconnects
49 }
50
51 int main() {
52     int server_fd = socket(AF_INET, SOCK_STREAM, 0);
53     int opt = 1;
54     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
55         sizeof(opt));
56
57     sockaddr_in addr{};
58     addr.sin_family = AF_INET;
59     addr.sin_port = htons(6379);
60     addr.sin_addr.s_addr = INADDR_ANY;
61
62     bind(server_fd, (sockaddr*)&addr, sizeof(addr));
63     listen(server_fd, 128); // large backlog for real
64                               usage
65     cout << "Redis server listening on :6379\n";
66
67     // Infinite accept loop --- one thread per client
68     while (true) {
69         int client_fd = accept(server_fd, nullptr, nullptr)
70         ;
71         if (client_fd < 0) continue;
72     }

```

```

66         // Detach: thread cleans itself up when done
67         thread(handleClient, client_fd).detach();
68     }
69     return 0;
70 }

```

6.2.1 Key Concepts — Threading

`thread(handleClient, client_fd).detach()`

- `thread(fn, arg)` — Creates a new OS thread running `fn(arg)`.
- `.detach()` — Tells the C++ runtime: "I won't join this thread; let it run independently and clean up itself."
- **Alternative:** `.join()` would block the main thread until the spawned thread finishes — useless here.

`unordered_map` and `mutex`

Race condition warning

`unordered_map` is **not thread-safe**. If two threads write simultaneously, the map can corrupt. We protect it with a `mutex`:

```

1  // Write operation
2  {
3      lock_guard<mutex> lock(store_mutex); // acquires
          lock
4      store[key] = value;
5  } // lock_guard destructor releases lock automatically
    (RAII)

```

`lock_guard` uses **RAII** (Resource Acquisition Is Initialization) — the lock is released automatically when `lock` goes out of scope, even if an exception is thrown.

Interview Q&A — Threading

Q: What is a race condition?

A: When two threads access shared data concurrently and at least one writes, producing unpredictable results that depend on thread scheduling.

Q: What is a mutex?

A: Mutual Exclusion — a synchronization primitive that only one thread can hold at a time. Other threads block at `lock()` until it's released.

Q: What is RAII?

A: A C++ idiom where a resource (lock, file, memory) is tied to an object's lifetime: acquired in the constructor, released in the destructor. This prevents leaks even when exceptions occur.

Chapter 7

Step 3 — Core Commands

7.1 Command Dispatcher

Command handler function

```
1  #include <algorithm>    // for transform (uppercase)
2
3  using namespace std;
4
5  // Global store defined in previous chapter
6  extern unordered_map<string, string> store;
7  extern mutex store_mutex;
8
9  // Helper: uppercase a string in-place
10 string toUpper(string s) {
11     transform(s.begin(), s.end(), s.begin(), ::toupper);
12     return s;
13 }
14
15 // Helper: build a RESP Simple String reply
16 string simpleStr(const string& msg) {
17     return "+" + msg + "\r\n";
18 }
19
20 // Helper: build a RESP Bulk String reply
21 string bulkStr(const string& data) {
22     return "$" + to_string(data.size()) + "\r\n" + data + "
23     \r\n";
24 }
25
26 // Helper: RESP Null Bulk String (key not found)
27 string nullBulk() {
28     return "$-1\r\n";
29 }
30
31 // Helper: RESP Integer reply
32 string intReply(long long n) {
33     return ":" + to_string(n) + "\r\n";
34 }
```



```

33 }
34
35 // Helper: RESP Error reply
36 string errReply(const string& msg) {
37     return "-ERR " + msg + "\r\n";
38 }
39
40 // Main command handler --- returns a RESP-encoded response
    string
41 string handleCommand(const vector<string>& args) {
42     if (args.empty()) return errReply("empty command");
43
44     string cmd = toUpper(args[0]);    // commands are case-
        insensitive
45
46     // -- PING
        -----
47     if (cmd == "PING") {
48         if (args.size() == 1) return simpleStr("PONG");
49         return bulkStr(args[1]);      // PING <message>
50     }
51
52     // -- ECHO
        -----
53     if (cmd == "ECHO") {
54         if (args.size() < 2) return errReply("wrong number
            of args");
55         return bulkStr(args[1]);
56     }
57
58     // -- SET
        -----
59     if (cmd == "SET") {
60         if (args.size() < 3) return errReply("wrong number
            of args");
61         lock_guard<mutex> lock(store_mutex);
62         store[args[1]] = args[2];
63         return simpleStr("OK");
64     }
65
66     // -- GET
        -----
67     if (cmd == "GET") {
68         if (args.size() < 2) return errReply("wrong number
            of args");
69         lock_guard<mutex> lock(store_mutex);
70         auto it = store.find(args[1]);
71         if (it == store.end()) return nullBulk();
72         return bulkStr(it->second);
73     }

```

```

74
75 // -- EXISTS
76 -----
77 if (cmd == "EXISTS") {
78     lock_guard<mutex> lock(store_mutex);
79     long long count = 0;
80     for (size_t i = 1; i < args.size(); i++) {
81         count += store.count(args[i]);
82     }
83     return intReply(count);
84 }
85
86 // -- DEL
87 -----
88 if (cmd == "DEL") {
89     lock_guard<mutex> lock(store_mutex);
90     long long deleted = 0;
91     for (size_t i = 1; i < args.size(); i++) {
92         deleted += store.erase(args[i]);
93     }
94     return intReply(deleted);
95 }
96
97 // -- INCR
98 -----
99 if (cmd == "INCR") {
100     if (args.size() < 2) return errReply("wrong number
101         of args");
102     lock_guard<mutex> lock(store_mutex);
103     auto& val = store[args[1]]; // inserts "" if
104         missing
105     long long n;
106     try { n = stoll(val); }
107     catch (...) { return errReply("not an integer"); }
108     val = to_string(++n);
109     return intReply(n);
110 }
111
112 return errReply("unknown command '" + args[0] + "'");
113 }

```

7.2 Command Explanations

7.2.1 PING

PING command

`PING` is the heartbeat command. With no argument: returns `+PONG\r\n`. With an argument (`PING hello`): echoes the argument back as a bulk string.

Use case: health checks — load balancers ping Redis to verify it is alive.

7.2.2 ECHO

Returns whatever argument you give it. Useful for testing the protocol layer — if `ECHO hello` returns `"hello"`, the connection and RESP parser are both working correctly.

7.2.3 SET & GET

- `store[args[1]] = args[2]` — Inserts or overwrites the key-value pair in the `unordered_map`. Amortised $O(1)$ time complexity.
- `store.find(args[1])` — Returns an iterator to the entry, or `store.end()` if not found. We check `it == store.end()` rather than using `operator[]` to avoid accidentally inserting empty strings.
- `nullBulk()` — Returns `$_1\r\n`, the RESP encoding for "key does not exist."

7.2.4 EXISTS

Takes one or more keys; returns an integer count of how many exist. We loop over all key arguments and sum up `store.count(key)` (returns 0 or 1).

7.2.5 DEL

Removes keys; returns how many were actually deleted. `unordered_map::erase(key)` returns 1 if the key existed, 0 otherwise.

7.2.6 INCR

1. `auto& val = store[args[1]]` — Gets a *reference* to the value. If the key is missing, `operator[]` inserts an empty string and we treat it as 0.
2. `stoll(val)` — Parse to `long long`. If it fails (non-numeric), return an error.
3. `val = to_string(++n)` — Increment and store back.
4. Return the new integer value.

Interview Q&A — Data Structures**Q: Why unordered_map and not map?**

A: `unordered_map` has $O(1)$ amortised lookup via hashing. `map` is a red-black tree with $O(\log n)$ lookup — slower for a cache that needs microsecond response times.

Q: What happens if two keys hash to the same bucket?

A: A **collision**. `unordered_map` handles it with *separate chaining* (a linked list per bucket in most implementations). Worst case degrades to $O(n)$, but it is rare with a good hash function.

Q: How does Redis actually store keys in production?

A: Redis uses a custom hash table with *incremental rehashing*: two hash tables exist during resize; keys are moved gradually to avoid a single big pause.

Chapter 8

Step 4 — Key Expiry (TTL)

8.1 Concept

Time-to-Live (TTL)

Redis allows keys to have an expiry time. After that time, the key behaves as if it does not exist. This is used for session tokens, rate-limiting counters, caches, and OTPs.

SET options for expiry:

- SET key value EX 10 — expire in 10 seconds
- SET key value PX 500 — expire in 500 milliseconds

8.2 Approach: Lazy Expiry

We use **lazy expiry**: we do not run a background timer thread for every key. Instead, we check expiry *at the moment of access*. If a key is expired, we delete it and return nil.

This is exactly how Redis does it (combined with a periodic sweep).

8.3 Implementation

TTL implementation with std::chrono

```
1 #include <chrono>
2 #include <optional>
3
4 using namespace std;
5 using namespace chrono;
6
7 // Expiry map: key -> absolute deadline (milliseconds since epoch)
8 unordered_map<string, long long> expiry_map;
9 // store and store_mutex from Chapter 6
```

```

10
11 // Get current time in milliseconds
12 long long nowMs() {
13     return duration_cast<milliseconds>(
14         steady_clock::now().time_since_epoch()
15     ).count();
16 }
17
18 // Check if key is expired (and delete it if so)
19 bool isExpired(const string& key) {
20     auto it = expiry_map.find(key);
21     if (it == expiry_map.end()) return false; // no TTL
22     set
23     if (nowMs() >= it->second) {
24         store.erase(key); // lazy delete from main
25         store
26         expiry_map.erase(it); // remove TTL entry too
27         return true;
28     }
29     return false;
30 }
31
32 // Enhanced SET handler (with EX/PX support)
33 string handleSet(const vector<string>& args) {
34     // args: ["SET", key, value, "EX"/"PX", duration]
35     if (args.size() < 3) return errReply("wrong number of
36     args");
37
38     const string& key = args[1];
39     const string& value = args[2];
40
41     lock_guard<mutex> lock(store_mutex);
42     store[key] = value;
43     expiry_map.erase(key); // clear any previous TTL
44
45     // Parse optional EX / PX flags
46     for (size_t i = 3; i + 1 < args.size(); i += 2) {
47         string flag = toUpper(args[i]);
48         long long amount = stoll(args[i + 1]);
49
50         if (flag == "EX") {
51             expiry_map[key] = nowMs() + amount * 1000;
52         } else if (flag == "PX") {
53             expiry_map[key] = nowMs() + amount;
54         }
55     }
56     return simpleStr("OK");
57 }
58
59 // Enhanced GET handler (with expiry check)

```

```

57 string handleGet(const vector<string>& args) {
58     if (args.size() < 2) return errReply("wrong number of
        args");
59
60     lock_guard<mutex> lock(store_mutex);
61     const string& key = args[1];
62
63     if (isExpired(key)) return nullBulk();    // key expired
64
65     auto it = store.find(key);
66     if (it == store.end()) return nullBulk();
67     return bulkStr(it->second);
68 }

```

8.4 Chrono — C++ Time Library

Type / Function	Meaning
<code>steady_clock</code>	Monotonic clock — never goes backwards, unaffected by NTP or daylight saving
<code>system_clock</code>	Wall-clock time — can jump; avoid for TTL
<code>now()</code>	Current time point
<code>time_since_epoch()</code>	Duration since the clock's epoch
<code>duration_cast<milliseconds>(...)</code>	Convert any duration to ms
<code>.count()</code>	Extract the raw integer from a duration

Why `steady_clock` not `system_clock`?

`system_clock` represents wall time. NTP (Network Time Protocol) can adjust it backwards. If wall time jumps back 30 seconds, keys would suddenly get 30 seconds of extra life. `steady_clock` is guaranteed *monotonically increasing* — ideal for measuring durations and deadlines.

Interview Q&A — TTL & Expiry

Q: How does Redis expire keys?

A: Two mechanisms: (1) **Lazy** — checked on access, expired key returns nil and is deleted. (2) **Active** — a background task samples random keys every 100ms and deletes expired ones. This prevents memory leaks from keys that are never accessed.

Q: What RESP type does `TTL` return for a non-existent key?

A: The integer `-2`. For a key with no TTL: `-1`. For a key with TTL remaining: remaining seconds.

Q: What is the `PERSIST` command?

A: Removes the TTL from a key, making it permanent.

Chapter 9

Step 5 — RDB Persistence

9.1 What is RDB?

RDB — Redis Database Snapshot

RDB is a binary file format that stores a point-in-time **snapshot** of all Redis data. On restart, Redis loads the RDB file to restore state. Snapshots are compacted and fast to load — ideal for backups and restarts.

RDB vs AOF:

	RDB	AOF (Append-Only File)
What	Snapshot of all data	Log of every write command
Size	Compact binary	Larger, grows over time
Load time	Fast	Slow (replays all commands)
Data loss	Up to last snapshot	Up to 1 second (fsync)
Default	Yes	Optional

9.2 RDB File Structure

RDB binary layout (simplified)

```
1 REDIS0011      <- Magic string + version (9 bytes)
2 0xFA          <- Auxiliary field opcode
3 ...          <- metadata (redis-ver, redis-bits, etc
   .)
4 0xFE          <- Database selector opcode
5 0x00          <- Database index (e.g. DB 0)
6 0xFB          <- Resize DB opcode
7 <varint>      <- hash table size
8 <varint>      <- expire hash table size
9 0x00          <- Value type: string
10 <key>        <- length-prefixed string
11 <value>      <- length-prefixed string
12 0xFC          <- Expire (milliseconds) opcode
```



```

13 <8 bytes LE>      <- unix timestamp ms
14 0x00             <- value type
15 <key> <value>     <- key and value
16 0xFF             <- EOF marker
17 <8 bytes>         <- CRC64 checksum

```

9.3 RDB Parser — C++ Implementation

RDB file reader

```

1  #include <fstream>
2  #include <cstdint>
3
4  using namespace std;
5
6  // Read a length-encoded integer from RDB (Redis length
   encoding)
7  size_t readLength(ifstream& f) {
8      uint8_t byte;
9      f.read((char*)&byte, 1);
10
11     // Top 2 bits determine encoding type
12     uint8_t enc = (byte & 0xC0) >> 6; // bits 7-6
13
14     if (enc == 0) {
15         return byte & 0x3F;           // 6-bit length
16     } else if (enc == 1) {
17         uint8_t next;
18         f.read((char*)&next, 1);
19         return ((byte & 0x3F) << 8) | next; // 14-bit
20     } else if (enc == 2) {
21         uint32_t val;
22         f.read((char*)&val, 4);
23         return __builtin_bswap32(val); // 32-bit big-endian
24     }
25     // enc == 3: special encoding (integers, compressed)
26     --- skip
27     return 0;
28 }
29
30 // Read a length-prefixed string from RDB
31 string readString(ifstream& f) {
32     size_t len = readLength(f);
33     string s(len, '\0');
34     f.read(&s[0], len);
35     return s;
36 }

```

```

37 // Load RDB file into the global store
38 void loadRDB(const string& path) {
39     ifstream f(path, ios::binary);
40     if (!f.is_open()) return; // no RDB file --- fresh
        start
41
42     // Skip header: "REDIS" (5) + version (4) = 9 bytes
43     f.seekg(9);
44
45     while (f.good()) {
46         uint8_t opcode;
47         f.read((char*)&opcode, 1);
48
49         if (opcode == 0xFA) {
50             // Auxiliary field --- skip key and value
                strings
51             readString(f);
52             readString(f);
53         } else if (opcode == 0xFE) {
54             // Database selector --- skip db index byte
55             readLength(f);
56         } else if (opcode == 0xFB) {
57             // Resize DB --- skip two length-encoded ints
58             readLength(f);
59             readLength(f);
60         } else if (opcode == 0xFF) {
61             break; // end of file
62         } else if (opcode == 0xFC) {
63             // Key with millisecond expiry
64             uint64_t expiry_ms;
65             f.read((char*)&expiry_ms, 8); // little-
                endian
66
67             uint8_t type;
68             f.read((char*)&type, 1); // value type
                (0=string)
69
70             string key = readString(f);
71             string val = readString(f);
72
73             lock_guard<mutex> lock(store_mutex);
74             store[key] = val;
75             expiry_map[key] = (long long)expiry_ms;
76         } else if (opcode == 0xFD) {
77             // Key with second expiry
78             uint32_t expiry_s;
79             f.read((char*)&expiry_s, 4);
80
81             uint8_t type;
82             f.read((char*)&type, 1);

```

```

83
84         string key = readString(f);
85         string val = readString(f);
86
87         lock_guard<mutex> lock(store_mutex);
88         store[key] = val;
89         expiry_map[key] = (long long)expiry_s * 1000;
90     } else if (opcode == 0x00) {
91         // Plain string key (no expiry)
92         string key = readString(f);
93         string val = readString(f);
94
95         lock_guard<mutex> lock(store_mutex);
96         store[key] = val;
97     }
98 }
99 }

```

9.4 RDB — Key Concepts

9.4.1 Length Encoding

Redis uses a compact **length-encoded integer** format:

Top 2 bits	Encoding	Meaning
00	6-bit	Length is in the remaining 6 bits (0–63)
01	14-bit	Read one more byte; 14 bits total (0–16383)
10	32-bit	Next 4 bytes are a big-endian 32-bit integer
11	Special	Integer encoded as a string (8/16/32-bit)

9.4.2 Endianness in RDB

- Expiry timestamps in RDB are stored in **little-endian** (least significant byte first).
- 32-bit lengths in type-2 encoding are **big-endian**. We use `__builtin_bswap32` to flip.
- x86/x86-64 CPUs are natively little-endian, so reading `uint64_t` directly gives the correct value on Linux/macOS.

9.5 Command-Line Arguments

The Codecrafters test harness starts your server with:

Parsing `-dir` and `-dbfilename`

```
1 // Global config
2 string rdb_dir      = "/tmp";      // default
3 string rdb_filename = "dump.rdb";  // default
4
5 int main(int argc, char* argv[]) {
6     // Parse: ./server --port 6380 --dir /tmp --dbfilename
        dump.rdb
7     for (int i = 1; i < argc - 1; i++) {
8         string flag = argv[i];
9         if (flag == "--port")      { /* handle port */ }
10        if (flag == "--dir")        rdb_dir = argv[i + 1];
11        if (flag == "--dbfilename") rdb_filename = argv[i +
12            1];
13    }
14
15    // Load RDB on startup
16    loadRDB(rdb_dir + "/" + rdb_filename);
17
18    // ... rest of main (socket setup, accept loop)
19 }
```

Interview Q&A — Persistence

Q: How does Redis avoid blocking while writing RDB?

A: Redis calls `fork()`. The child process gets a copy-on-write snapshot of memory and writes RDB to disk. The parent continues serving clients without pausing.

Q: What is copy-on-write?

A: After `fork()`, parent and child share the same physical memory pages. Only when one process writes a page does the OS create a private copy — minimising memory overhead.

Q: What is CRC64 in RDB?

A: A 64-bit cyclic redundancy check appended at the end of the RDB file. On load, Redis recomputes the checksum and rejects the file if it differs — detecting corruption.

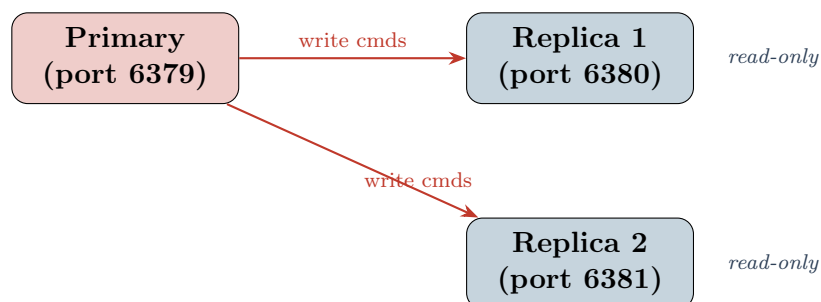
Chapter 10

Step 6 — Replication

10.1 Replication Overview

Leader–Follower Replication

Redis replication is **asynchronous, leader–follower**: one **primary** (master) accepts writes; one or more **replicas** (slaves) receive a copy of every write command and maintain an identical dataset. Replicas can serve reads, improving throughput.



10.2 Replication Handshake

When a replica starts, it connects to the primary and performs:

Replica connects to primary -- sequence

```
1 # Step 1: Replica -> Primary
2 PING
3 # Primary replies: +PONG
4
5 # Step 2: Replica -> Primary
6 REPLCONF listening-port <replica-port>
7 # Primary replies: +OK
8
9 # Step 3: Replica -> Primary
10 REPLCONF capa psync2
11 # Primary replies: +OK
```

```

12
13 # Step 4: Replica -> Primary
14 PSYNC <replication-id> <offset>
15 # If fresh: PSYNC ? -1
16 # Primary replies: +FULLRESYNC <replid> 0\r\n
17 # Followed by: RDB file bulk transfer

```

10.3 Replication ID and Offset

Concept	Meaning
Replication ID	40-character alphanumeric ID generated when a primary starts. Identifies the replication stream.
Offset	Number of bytes of commands the replica has processed. Starts at 0; increments as write commands are propagated. Primary tracks this per replica.

10.4 Primary-Side Implementation

Primary tracks replicas

```

1  #include <vector>
2  #include <mutex>
3
4  using namespace std;
5
6  // Each connected replica's write socket
7  vector<int> replica_fds;
8  mutex replica_mutex;
9
10 // Generate a random 40-char replication ID
11 string generateReplID() {
12     const string chars =
13         "0123456789abcdefghijklmnopqrstuvwxyz"
14         "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
15     string id(40, ' ');
16     srand((unsigned)time(nullptr));
17     for (char& c : id) c = chars[rand() % chars.size()];
18     return id;
19 }
20
21 string repl_id      = generateReplID();
22 long long repl_offset = 0;    // bytes propagated
23
24 // Called from handleCommand when SET/DEL/etc. is executed
25 void propagateToReplicas(const string& raw_cmd) {
26     lock_guard<mutex> lk(replica_mutex);

```

```

27     for (int fd : replica_fds) {
28         send(fd, raw_cmd.c_str(), raw_cmd.size(), 0);
29     }
30     repl_offset += (long long)raw_cmd.size();
31 }
32
33 // Handle INFO replication sub-command
34 string handleInfo(const vector<string>& args) {
35     string role = "master"; // or "slave" if we are a
        replica
36     string info =
37         "# Replication\r\n"
38         "role:" + role + "\r\n"
39         "master_replid:" + repl_id + "\r\n"
40         "master_repl_offset:" + to_string(repl_offset) + "\r\n";
41     return bulkStr(info);
42 }

```

10.5 Replica-Side: Connecting to Primary

Replica connects to primary on startup

```

1  #include <arpa/inet.h>
2
3  using namespace std;
4
5  // Connect to primary and perform handshake
6  int connectToPrimary(const string& primary_host,
7                      int primary_port,
8                      int my_port) {
9      // 1. Create socket and connect
10     int fd = socket(AF_INET, SOCK_STREAM, 0);
11
12     sockaddr_in addr{};
13     addr.sin_family = AF_INET;
14     addr.sin_port = htons(primary_port);
15     inet_pton(AF_INET, primary_host.c_str(), &addr.sin_addr);
16
17     if (connect(fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
18         cerr << "Cannot connect to primary\n";
19         return -1;
20     }
21
22     // Helper: send a RESP command and ignore reply
23     auto sendCmd = [&](const string& cmd) {
24         send(fd, cmd.c_str(), cmd.size(), 0);

```

```

25         char buf[256]; recv(fd, buf, sizeof(buf), 0);
26     };
27
28     // 2. Handshake steps
29     // PING
30     sendCmd("*1\r\n$4\r\nPING\r\n");
31
32     // REPLCONF listening-port <port>
33     string port_str = to_string(my_port);
34     sendCmd("*3\r\n$8\r\nREPLCONF\r\n"
35            "$14\r\nlistening-port\r\n"
36            "$" + to_string(port_str.size()) + "\r\n"
37            + port_str + "\r\n");
38
39     // REPLCONF capa psync2
40     sendCmd("*3\r\n$8\r\nREPLCONF\r\n"
41            "$4\r\ncapa\r\n$6\r\npsync2\r\n");
42
43     // PSYNC ? -1 (full resync request)
44     sendCmd("*3\r\n$5\r\nPSYNC\r\n$1\r\n?\r\n$2\r\n-1\r\n")
45         ;
46
47     // 3. Receive full RDB from primary (skip it for basic impl.)
48     // In a full impl., parse the $<N>\r\n<rdb-bytes> bulk string
49
50     return fd; // keep this fd open --- commands stream on it
51 }

```

Interview Q&A — Replication

Q: Is Redis replication synchronous or asynchronous?

A: **Asynchronous** by default. The primary replies to the client before waiting for replicas to acknowledge. This means replicas may lag slightly — called *replication lag*.

Q: What is WAIT command?

A: Forces the primary to block until at least N replicas have acknowledged all prior write commands. Provides stronger consistency guarantees.

Q: What happens if the primary fails?

A: A replica can be promoted to primary using `REPLICAOF NO ONE`. Redis Sentinel automates this. Redis Cluster handles this natively.

Chapter 11

Step 7 — Complete Server

11.1 Final server.cpp

server.cpp -- production-ready skeleton

```
1  //
2  // -----
3  //  Redis Clone --- Complete Server
4  //  Combines: TCP, RESP parser, commands, TTL, RDB,
5  //  replication
6  // -----
7
8  #include <iostream>
9  #include <string>
10 #include <vector>
11 #include <unordered_map>
12 #include <mutex>
13 #include <thread>
14 #include <chrono>
15 #include <algorithm>
16 #include <stdexcept>
17 #include <fstream>
18 #include <cstring>
19 #include <cstdint>
20 #include <sys/socket.h>
21 #include <netinet/in.h>
22 #include <arpa/inet.h>
23 #include <unistd.h>
24
25 using namespace std;
26 using namespace chrono;
27
28 // -- Global state
29 // -----
30 unordered_map<string, string> store;
```

```

27 unordered_map<string, long long> expiry_map;
28 mutex                          store_mutex;
29
30 vector<int> replica_fds;
31 mutex      replica_mutex;
32
33 string rdb_dir      = "/tmp";
34 string rdb_filename = "dump.rdb";
35 int    server_port  = 6379;
36
37 string master_host  = "";
38 int    master_port  = -1;
39 bool   is_replica   = false;
40
41 // -- Time helper
42 -----
43 long long nowMs() {
44     return duration_cast<milliseconds>(
45         steady_clock::now().time_since_epoch()).count();
46 }
47
48 // -- RESP builders
49 -----
50 string simpleStr(const string& s){ return "+" + s + "\r\n";
51 }
52 string errReply (const string& s){ return "-ERR " + s + "\r\n"; }
53 string nullBulk ()                { return "$-1\r\n"; }
54 string intReply (long long n)     { return ":" + to_string(n
55     ) + "\r\n"; }
56 string bulkStr  (const string& s){
57     return "$" + to_string(s.size()) + "\r\n" + s + "\r\n";
58 }
59
60 // -- Utility
61 -----
62 string toUpper(string s) {
63     transform(s.begin(), s.end(), s.begin(), ::toupper);
64     return s;
65 }
66
67 // -- Expiry check (call under store_mutex)
68 -----
69 bool isExpired(const string& key) {
70     auto it = expiry_map.find(key);
71     if (it == expiry_map.end()) return false;
72     if (nowMs() >= it->second) {
73         store.erase(key);
74         expiry_map.erase(it);
75         return true;
76     }
77 }

```

```

70     }
71     return false;
72 }
73
74 // -- RESP parser
75 -----
76 vector<string> parseRESP(const string& raw) {
77     vector<string> args;
78     size_t pos = 0;
79     if (raw.empty() || raw[pos] != '*') return args;
80     pos++;
81     size_t crlf = raw.find("\r\n", pos);
82     if (crlf == string::npos) return args;
83     int count = stoi(raw.substr(pos, crlf - pos));
84     pos = crlf + 2;
85     for (int i = 0; i < count && pos < raw.size(); i++) {
86         if (raw[pos] != '$') break;
87         pos++;
88         crlf = raw.find("\r\n", pos);
89         if (crlf == string::npos) break;
90         int len = stoi(raw.substr(pos, crlf - pos));
91         pos = crlf + 2;
92         args.push_back(raw.substr(pos, len));
93         pos += len + 2;
94     }
95     return args;
96 }
97
98 // -- Command handler
99 -----
100 string handleCommand(const vector<string>& args, bool
    from_replica = false);
101
102 // -- Propagate write to all replicas
103 -----
104 void propagate(const string& raw) {
105     lock_guard<mutex> lk(replica_mutex);
106     for (int fd : replica_fds)
107         send(fd, raw.c_str(), raw.size(), 0);
108 }
109
110 // -- Client handler thread
111 -----
112 void handleClient(int cfd) {
113     char buf[4096];
114     string leftover; // buffer for partial
115                     reads
116
117     while (true) {
118         ssize_t n = recv(cfd, buf, sizeof(buf) - 1, 0);

```

```

114         if (n <= 0) break;
115         buf[n] = '\0';
116         leftover += string(buf, n);
117
118         // Process all complete commands in the buffer
119         while (true) {
120             if (leftover.empty()) break;
121             // Find the end of this command (naive: try
122             // parsing)
123             try {
124                 auto args = parseRESP(leftover);
125                 if (args.empty()) break;
126
127                 string cmd = toUpper(args[0]);
128                 string reply = handleCommand(args);
129                 send(cfd, reply.c_str(), reply.size(), 0);
130                 leftover.clear(); // simple: clear after
131                                 // each cmd
132             } catch (...) { break; }
133         }
134     }
135     close(cfd);
136 }
137
138 string handleCommand(const vector<string>& args, bool
139 from_replica) {
140     if (args.empty()) return errReply("empty command");
141     string cmd = toUpper(args[0]);
142
143     if (cmd == "PING") {
144         return args.size() > 1 ? bulkStr(args[1]) :
145             simpleStr("PONG");
146     }
147
148     if (cmd == "ECHO") {
149         return args.size() > 1 ? bulkStr(args[1])
150             : errReply("wrong number of
151                 args");
152     }
153
154     if (cmd == "SET") {
155         if (args.size() < 3) return errReply("wrong number
156             of args");
157         lock_guard<mutex> lk(store_mutex);
158         store[args[1]] = args[2];
159         expiry_map.erase(args[1]);
160         for (size_t i = 3; i + 1 < args.size(); i += 2) {
161             string flag = toUpper(args[i]);
162             long long amt = stoll(args[i + 1]);
163             if (flag == "EX") expiry_map[args[1]] =
164                 nowMs() + amt * 1000;
165         }
166     }
167 }

```

```

156         else if (flag == "PX") expiry_map[args[1]] =
            nowMs()+amt;
157     }
158     if (!from_replica) {
159         // Build raw RESP to propagate
160         string raw = "*" + to_string(args.size()) + "\r\n";
161         for (auto& a : args)
162             raw += "$" + to_string(a.size()) + "\r\n" +
                a + "\r\n";
163         propagate(raw);
164     }
165     return simpleStr("OK");
166 }
167 if (cmd == "GET") {
168     if (args.size() < 2) return errReply("wrong number
        of args");
169     lock_guard<mutex> lk(store_mutex);
170     if (isExpired(args[1])) return nullBulk();
171     auto it = store.find(args[1]);
172     return it == store.end() ? nullBulk() : bulkStr(it
        ->second);
173 }
174 if (cmd == "DEL") {
175     lock_guard<mutex> lk(store_mutex);
176     long long d = 0;
177     for (size_t i = 1; i < args.size(); i++) d += store
        .erase(args[i]);
178     return intReply(d);
179 }
180 if (cmd == "EXISTS") {
181     lock_guard<mutex> lk(store_mutex);
182     long long c = 0;
183     for (size_t i = 1; i < args.size(); i++) c += store
        .count(args[i]);
184     return intReply(c);
185 }
186 if (cmd == "INCR") {
187     if (args.size() < 2) return errReply("wrong number
        of args");
188     lock_guard<mutex> lk(store_mutex);
189     auto& v = store[args[1]];
190     long long n = 0;
191     if (!v.empty()) try { n = stoll(v); }
192                     catch (...) { return errReply("not
        an integer"); }
193     v = to_string(++n);
194     return intReply(n);
195 }
196 if (cmd == "KEYS") {

```

```

197     lock_guard<mutex> lk(store_mutex);
198     // args[1] is pattern ("*" = all) --- simplified:
199     // return all
200     vector<string> keys;
201     for (auto& [k, _] : store) {
202         if (!isExpired(k)) keys.push_back(k);
203     }
204     string reply = "*" + to_string(keys.size()) + "\r\n";
205     for (auto& k : keys)
206         reply += "$" + to_string(k.size()) + "\r\n" + k
207         + "\r\n";
208     return reply;
209 }
210 if (cmd == "TYPE") {
211     if (args.size() < 2) return errReply("wrong number
212     of args");
213     lock_guard<mutex> lk(store_mutex);
214     if (isExpired(args[1])) return simpleStr("none");
215     return store.count(args[1]) ? simpleStr("string")
216     : simpleStr("none");
217 }
218 if (cmd == "REPLCONF") return simpleStr("OK");
219 if (cmd == "PSYNC") {
220     // Return FULLRESYNC + send empty RDB
221     string repl_id = "8371
222     b4fb1155b71f4a04d3e1bc3e18c4a990aeeb";
223     string hdr = "+FULLRESYNC " + repl_id + " 0\r\n";
224     // Minimal valid empty RDB (hex-decoded)
225     static const char rdb[] = "\x52\x45\x44\x49\x53\x30
226     \x30\x31"
227     "\x31\xfa\x09\x72\x65\
228     x64\x69\x73"
229     "\x2d\x76\x65\x72\x05\
230     x37\x2e\x32"
231     "\x2e\x30\xff\x0a\x25\
232     x42\x38\xbe"
233     "\xa9\x0c\xef";
234     string body(rdb, sizeof(rdb) - 1);
235     return hdr + "$" + to_string(body.size()) + "\r\n"
236     + body;
237 }
238 return errReply("unknown command '" + args[0] + "'");
239 }
240
241 int main(int argc, char* argv[]) {
242     // Parse CLI args
243     for (int i = 1; i < argc; i++) {
244         string a = argv[i];

```

```
236         if (a == "--port"          && i+1 < argc) server_port
           = stoi(argv[++i]);
237         if (a == "--dir"           && i+1 < argc) rdb_dir
           = argv[++i];
238         if (a == "--dbfilename" && i+1 < argc) rdb_filename
           = argv[++i];
239         if (a == "--replicaof"    && i+1 < argc) {
240             is_replica = true;
241             master_host = argv[++i];
242             master_port = stoi(argv[++i]);
243         }
244     }
245
246     // Setup server socket
247     int sfd = socket(AF_INET, SOCK_STREAM, 0);
248     int opt = 1;
249     setsockopt(sfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(
        opt));
250
251     sockaddr_in addr{};
252     addr.sin_family      = AF_INET;
253     addr.sin_port        = htons(server_port);
254     addr.sin_addr.s_addr = INADDR_ANY;
255
256     bind(sfd, (sockaddr*)&addr, sizeof(addr));
257     listen(sfd, 128);
258
259     cout << "Redis server on :" << server_port << "\n";
260
261     // Accept loop
262     while (true) {
263         int cfd = accept(sfd, nullptr, nullptr);
264         if (cfd < 0) continue;
265         thread(handleClient, cfd).detach();
266     }
267 }
```

Chapter 12

Interview Preparation

12.1 System Design Questions

Q: Design a distributed cache

Answer framework:

1. **Requirements:** Read/write ratio, latency SLA, data size, consistency needs.
2. **Core:** In-memory hash map with TTL.
3. **Scaling:** Consistent hashing to distribute keys across nodes (Redis Cluster uses 16,384 hash slots).
4. **Availability:** Primary-replica replication. Redis Sentinel monitors and auto-promotes replicas.
5. **Persistence:** RDB snapshots for fast restart; AOF for durability.
6. **Eviction:** LRU or LFU policy when memory is full (Redis has 8 eviction policies).

Q: How does Redis achieve microsecond latency?

1. All data in RAM — no disk I/O on reads.
2. Single-threaded event loop — no lock contention (in classic Redis; Redis 6+ has I/O threads).
3. Efficient protocol (RESP) — minimal parsing overhead.
4. Optimised data structures (SDS strings, zip lists, skip lists for sorted sets).
5. No virtual memory — data fits in RAM by design.

Q: Explain the CAP theorem and where Redis fits

CAP Theorem: A distributed system can guarantee at most two of:

- **Consistency** — every read sees the latest write
- **Availability** — every request gets a response
- **Partition tolerance** — system works despite network splits

Redis with Sentinel: AP system — prefers availability. Asynchronous replication means replicas may be slightly stale.

Redis with WAIT: approaches CP — blocks until replicas are caught up,

sacrificing availability.

12.2 C++ Systems Programming Questions

Q: What is the difference between stack and heap?

	Stack	Heap
Allocation	Automatic (local vars)	Manual (<code>new</code> / <code>malloc</code>)
Speed	Very fast (just decrement SP)	Slower (allocator overhead)
Size	Limited (~8 MB default)	Limited only by RAM
Lifetime	Until scope ends	Until <code>delete</code> / <code>free</code>
Thread	Each thread has its own	Shared between threads

Q: What is a file descriptor?

A non-negative integer the kernel uses to represent an open resource: file, socket, pipe, or device. Returned by `open()`, `socket()`, `accept()`. The kernel maintains a table mapping FD numbers to internal structures.

Standard FDs: 0 = stdin, 1 = stdout, 2 = stderr. Our `server_fd` and `client_fd` are ≥ 3 .

Q: Explain `send()` vs `write()`

Both write bytes to a file descriptor. `send()` is socket-specific and accepts flags (e.g., `MSG_NOSIGNAL` to suppress SIGPIPE). `write()` is generic. For sockets, prefer `send()`.

Important: neither guarantees all bytes are sent in one call. Always loop until all bytes are sent:

```

1 void sendAll(int fd, const string& data) {
2     size_t sent = 0;
3     while (sent < data.size()) {
4         ssize_t n = send(fd, data.c_str() + sent,
5                           data.size() - sent, 0);
6         if (n <= 0) break;
7         sent += n;
8     }
9 }
```

Q: What is an epoll and why is it better than select?

Both are I/O multiplexing mechanisms — they let one thread monitor many file descriptors simultaneously.

- `select()` — passes all FDs on every call; $O(n)$ scan; limited to 1024 FDs.
- `poll()` — similar to select but no FD limit; still $O(n)$.

- `epoll()` (Linux only) — kernel maintains a list; only *ready* FDs are returned; $O(1)$ per event. Scales to 100,000+ connections.

Production Redis uses `epoll` on Linux, `kqueue` on macOS — our project uses threads instead for simplicity.

12.3 Data Structure Questions

Q: Implement a simple LRU cache in C++

```
1  #include <list>
2  #include <unordered_map>
3  using namespace std;
4
5  class LRUCache {
6      int cap;
7      list<pair<int,int>> lst;    // {key, value}, front =
                                MRU
8      unordered_map<int, list<pair<int,int>>::iterator> mp
9      ;
10 public:
11     LRUCache(int capacity) : cap(capacity) {}
12
13     int get(int key) {
14         auto it = mp.find(key);
15         if (it == mp.end()) return -1;
16         lst.splice(lst.begin(), lst, it->second); //
                                move to front
17         return it->second->second;
18     }
19
20     void put(int key, int val) {
21         if (mp.count(key)) {
22             lst.erase(mp[key]);
23         } else if ((int)lst.size() == cap) {
24             mp.erase(lst.back().first); // evict LRU
25             lst.pop_back();
26         }
27         lst.push_front({key, val});
28         mp[key] = lst.begin();
29     };
30 }
```

This is **LeetCode 146** — extremely common in interviews alongside Redis discussions.

Chapter 13

Concepts Glossary

Term	Definition
AOF	Append-Only File — persistence mode that logs every write command. Replaying the log restores data.
CRLF	Carriage Return + Line Feed (<code>\r\n</code>). RESP uses CRLF as the message delimiter.
Endianness	The order in which multi-byte integers are stored. Little-endian (x86): LSB first. Big-endian (network): MSB first.
Event Loop	A loop that repeatedly calls <code>epoll</code> / <code>kqueue</code> to find ready sockets, processes them, and loops back. Redis is single-threaded around its event loop.
fd	File Descriptor — integer handle to an open OS resource (file, socket, pipe).
Hash Slot	Redis Cluster divides the key space into 16,384 slots. Each key maps to one slot via CRC16.
INADDR_ANY	Special IPv4 address <code>0.0.0.0</code> meaning "bind to all network interfaces."
Lazy Expiry	Expiry checked on access, not proactively. Expired key is deleted when first accessed.
LFU	Least Frequently Used — eviction policy that removes keys accessed the fewest times.
LRU	Least Recently Used — eviction policy that removes the key not accessed for the longest time.
Monotonic clock	A clock that never goes backwards — ideal for measuring durations (<code>steady_clock</code>).
Mutex	Mutual Exclusion lock — ensures only one thread accesses shared data at a time.
NTP	Network Time Protocol — synchronises wall clock. Can adjust time backwards — unsafe for durations.
Pipelining	Sending multiple RESP commands without waiting for replies — reduces round-trip latency.
RAII	Resource Acquisition Is Initialization — C++ pattern where resource lifetime = object lifetime.
Race Condition	Bug where output depends on unpredictable thread scheduling. Prevented with mutexes.

RDB	Redis Database Snapshot — compact binary file of all keys at a point in time.
Replication Lag	Delay between primary writing a command and a replica applying it.
RESP	Redis Serialization Protocol — text-based binary-safe protocol for client-server communication.
Sentinel	Redis high-availability system — monitors primary, auto-promotes replica on failure.
SO_REUSEADDR	Socket option that allows a server to rebind to a port immediately after restart.
TCP	Transmission Control Protocol — reliable, ordered, connection-oriented byte stream protocol.
TIME_WAIT	TCP state after a connection closes — OS holds the port for 60s to catch delayed packets.
TTL	Time To Live — duration until a key expires.
UDP	User Datagram Protocol — unreliable, connectionless, faster than TCP. Not used by Redis.

Chapter 14

Cheat Sheets

14.1 RESP Encoding Reference

RESP encoding cheat sheet

```
1  # Simple String (no \r\n inside)
2  +OK\r\n
3  +PONG\r\n
4
5  # Error
6  -ERR unknown command\r\n
7
8  # Integer
9  :42\r\n
10 :-1\r\n
11
12 # Bulk String
13 $5\r\nhello\r\n
14 $-1\r\n          <- null (key not found)
15
16 # Array
17 *3\r\n$3\r\nSET\r\n$3\r\nfoo\r\n$3\r\nbar\r\n
18 *0\r\n          <- empty array
19 *-1\r\n         <- null array
```

14.2 Redis Commands — Quick Reference

Command	Syntax	Returns
PING	PING [msg]	PONG or msg
ECHO	ECHO msg	msg
SET	SET k v [EX n] [PX n]	OK
GET	GET k	value or nil
DEL	DEL k [k...]	integer (deleted count)
EXISTS	EXISTS k [k...]	integer (count found)

INCR	INCR k	integer (new value)
KEYS	KEYS pattern	array of matching keys
TYPE	TYPE k	string / list / set / none
TTL	TTL k	seconds remaining (-1/-2)
PTTL	PTTL k	milliseconds remaining
PERSIST	PERSIST k	1 if TTL removed, 0 otherwise
INFO	INFO [section]	bulk string of server info
CONFIG	CONFIG GET param	key-value pairs
REPLCONF	REPLCONF ...	OK (replication)
PSYNC	PSYNC replid offset	+FULLRESYNC ...
WAIT	WAIT numreplicas ms	integer (replicas in sync)

14.3 Build & Test Commands

Development workflow

```

1  # Build
2  cd build && cmake .. -DCMAKE_BUILD_TYPE=Debug && make -j4
3
4  # Run server
5  ./server --port 6379 --dir /tmp --dbfilename dump.rdb
6
7  # Run as replica
8  ./server --port 6380 --replicaof 127.0.0.1 6379
9
10 # Quick tests
11 redis-cli ping                # PONG
12 redis-cli set foo bar         # OK
13 redis-cli get foo             # "bar"
14 redis-cli set temp 1 ex 5     # OK (expires in 5s)
15 redis-cli ttl temp            # 4 (or similar)
16 redis-cli info replication    # show replication state
17 redis-cli keys "*"            # list all keys
18
19 # Protocol-level test (see raw RESP)
20 redis-cli --no-auth-warning -3 ping
21
22 # Load test (10k requests, 50 parallel)
23 redis-benchmark -n 10000 -c 50 -t ping,set,get

```

14.4 Common Errors and Fixes

Error	Fix
Address already in use	Add <code>SO_REUSEADDR</code> option, or kill <code>\$(lsof -ti:6379)</code>
Connection refused	Server not running, or wrong port

Segmentation fault	Accessing <code>nullptr</code> ; run with <code>gdb ./server</code> and check backtrace
double free or corruption	Two threads deleting same pointer; add mutex
stoi: invalid argument	Empty string or non-numeric RESP length; add try-catch
Broken pipe	Client disconnected before you finished sending; use <code>MSG_NOSIGNAL</code>

Chapter A

Debugging with GDB

GDB quick reference

```
1  # Compile with debug symbols
2  g++ -g -O0 -std=c++17 src/server.cpp -o server
3
4  # Start GDB
5  gdb ./server
6
7  # GDB commands
8  (gdb) run                # start program
9  (gdb) break handleCommand # set breakpoint at function
10 (gdb) break server.cpp:42 # set breakpoint at line 42
11 (gdb) continue           # continue to next
    breakpoint
12 (gdb) next               # step over
13 (gdb) step               # step into
14 (gdb) print args[0]      # print variable
15 (gdb) backtrace          # show call stack on crash
16 (gdb) info threads       # show all threads
17 (gdb) thread 2           # switch to thread 2
```


Chapter B

Valgrind — Memory Error Detection

Valgrind usage

```
1  # Install
2  sudo apt install valgrind
3
4  # Check for memory leaks and errors
5  valgrind --leak-check=full \
6           --show-leak-kinds=all \
7           --track-origins=yes \
8           ./server
9
10 # Check for data races (Helgrind)
11 valgrind --tool=helgrind ./server
```

Chapter C

Further Reading

- Redis official documentation
<https://redis.io/docs/>
- RESP protocol spec
<https://redis.io/docs/reference/protocol-spec/>
- RDB file format
https://rdb.fnordig.de/file_format.html
- Beej's Guide to Network Programming
<https://beej.us/guide/bgnet/>
- C++ Reference
<https://en.cppreference.com/>
- Codecrafters Redis challenge
<https://codecrafters.io/challenges/redis>
- "Designing Data-Intensive Applications" — Kleppmann
Chapters 5 (Replication) and 6 (Partitioning)

You are now ready

By working through this guide you have:

- Built a TCP server from raw socket calls
- Implemented a binary protocol parser (RESP)
- Created a concurrent key-value store with TTL
- Parsed a binary file format (RDB)
- Implemented leader-follower replication
- Prepared answers for Redis, C++, and systems-design interview questions

Push every stage to Codecrafters and watch the green checkmarks appear. Good luck!